

TPL Signum - RF Intruder Detection System

SIGMOD5TST

Demo Kits - 2.4Ghz 10-Ellipse 5-Hops

AT-Commands and TPL-CLI Manual

Version 0.8

1. Technology Introduction	4
2. SIGMOD5TST Demo Kits	4
3. Installation in the field	6
4. TPL-CLI manual human-friendly interface	6
5. AT-Commands interface	8
6. Intruder Detection Events	9
Over TPL-CLI	9
Over AT-Commands - #EVTDETECT	10
Events emulation for PC GUI testing	11
Detection from communication drop	13
Over TPL-CLI	13
Over AT-Commands - #EVTDETCOMM	13
7. Manual Sensitivity Configuration Commands	14
Over TPL-CLI	14
a. Threshold command	15
b. Gain command	16
Over AT-Commands	16
Threshold - #SETDETTTHR	16
Gain - #SETDETGAIN	16
8. Configuring detection modes	16
Set Detection Mode - #SETDETMODE	17
Request the current detection mode - #REQDETMODE	17
Detection Mode Event - #EVTDETMODE	17
9. Retrieving system status - Map command	18
Over TPL-CLI	18
Over AT-Command	19
Request Mesh Map - #REQMESHMAP	19
Device information event - #EVTMESHMAPDEV	20
Link information event - #EVTMESHMAPDEVLINK	20
Example	21
TEMPORARY - Get Mesh Map - #GETMESHMAP	21

Device information - #MESHMAPDEV	21
Link information - #MESHMAPDEVLINK	22
Example	22
10. Check communication over mesh - Ping command	23
Over TPL-CLI	23
Over AT-Command	23
PING Command transmission over the network - #PING	23
Example	24
11. Link-Up/Link-Down events	24
Over TPL-CLI	24
Over AT-Commands	25
Link established event - #EVTMESHLINKUP	25
Link dropped event - #EVTMESHLINKDOWN	25
12. Antenna selection	26
Set selection of current active antenna - #SETACTANT	26
Request current selected active antenna - #REQACTANT	26
Selected active antenna report event - #EVTACTANT	27
13. DFU - Firmware update through phone	27
nRF Connect for Mobile	27
1. Entering DFU mode:	27
Over TPL-CLI	27
2. Connecting over BLE	28
3. Transferring application file	29
14. Pure mesh commands - not fully operational in this released firmware	33
Mesh Commands	33
Send message – #MESHSEND	33
Mesh Events	33
Mesh received data event - #EVTMESHRCV	33
Mesh Settings commands	34
Create Network ID and Security - #CREATENETSEC	34
Get Network ID list - #GETNETSEC	34
Select Current Network ID and Security for TX - #SETTXNETSEC	34
Delete Network ID and Security - #DELNETSEC	35
Set VNet Virtual Network - #SETVNET	35
Set mesh device ID - #SETID	36
Add mesh device ID for receiving - #ADDRRCVID	36
Get mesh device IDs for receiving - #GETRCVID	36
Delete mesh device ID for receiving - #DELRCVID	37
Set TTL level - #SETTTL	37
Set Holdout Time level - #SETHOLDOUTTIME	37
Set Switches Per Message (internal) - #SETTSWPERMSG	38

Set Counts per Switch (internal) - #SETCNTPERSWITCH	39
Set Dynamic Params (internal) - #DYNSETPARAMS	39
Set priority #SETPRIORITY	39
Save current settings #SAVESETTINGS	40
Load settings #LOADSETTINGS	40
Delete settings #DELSETTINGS	40
Print settings list #PRTSETTINGS	40
Management Commands	40
Init (not necessary) - #INIT	41
Get version - #GETVERSION	41
Reset the device - #RESET	41
Set print level - #SETPRTLVL	42
Start cli uart - #TPLCLIUARTSTART	42
Start cli uart - #TPLCLISTART	42
Show all AT_CMD commands- #HELP	43
Get mac address device - #GETMAC	43
Echo the atcmd- #ECHO	43
Configure Baudrate - #BAUD	43
Management Events	44
Device Error event - #EVTERR	44
Trace print event - #EVTTRACE	44
Commands for Firmware Update (DFU) over AT-Commands	44
Prepare For Update - #PREPDFU	44
Prepare for update complete unsolicited event - #EVTPREPDFUCOMPLETE	45
Write DFU Data - #WRITEDFUDATA	45
Write DFU Data - #WRITEDFUDATAASYNC	46
Write DFU Data complete event - #EVTWRITEDFUDATA	46
Read DFU Data - #READDFUDATA	47
Read DFU Data - #READDFUDATAASYNC	47
Read DFU Data complete event - #EVTREADDFUDATA	48
Check Update package - #CHKDFUPKG	48
Check Update package complete event - #EVTCHKDFUPKG	49
Execute DFU - #EXEDFU	49
AT-Commands full update flow sequence diagram	50
Debug Commands	51
Mesh forward message event - #EVTFWDMMSG	51
Start simulating unsolicited messages - #STARTSIMRCVMSG	52
Stop simulating unsolicited responses - #STOPSIMRCVMSG	53
Commands for load status over AT-Commands	53
Get state of main transmitting storage - #HEAPGETSTAT	53

1. Technology Introduction

Our company, TechPack Lab, abbreviated as TPL, has invented and developed a unique intruder detection technology and system, called Signum. The Signum technology is based on RF, but unlike RF technologies like radar, our system doesn't have directivity, and detection units can be located anywhere and in any orientation. Furthermore, the units are extremely small, extremely low-power, and very low-cost.

The Signum technology is deployed together with another unique technology of ours, the TPL-Mesh. TPL-Mesh is an extremely optimized Mesh wireless communication. It allows participating units, like the Signum units, to be active and relaying in mesh for years at minimal batteries. The TPL-Mesh units can come in various throughput levels, from highest HD video-grade level to lowest levels as incorporated in miniature Signum units. While incorporated into Signum units it efficiently shares the same processor and transceiver, thus allowing extremely efficient sizes (down to 1.5x1.5cm PCBs), extremely low power (well under 1mA) and very accessible pricing for very high quantity deployments.

The Signum units come in multiple form factors, operating frequencies and management methods. Coupled with TechPack Lab's design services capabilities, these units can be easily adapted to any required use case.

2. SIGMOD5TST Demo Kits

The release of SIGMOD5TST contains 3 kits, with mostly identical software and API. These are the differences between the kits:

1. Kit with large evaluation board, transmitting at up to 20dBm
 - a. The PC connecting unit is a similar unit, just with USB cable to a PC
2. Kit with rather small custom board, transmitting at up to 8dBm
 - a. Board has hardware preparations for sound recording, which is not operational in the provided firmware
 - b. The PC connecting unit is demonstrated as nice enclosed USB dongle
3. Kit with modular custom boards set, allowing transmission at up to 20dBm
 - a. Kit contains 3 kind of boards:
 - i. Main detection minimized board, which can also be integrated into additional enclosures.
 - ii. Example of power board with batteries.
 - iii. Example of interfaces board with USB connector, few LEDs and buttons.
 - b. Board has hardware of precise seismic sensor, which is not operational in the provided firmware
 - c. Board supports long onboard recording of raw data for further analysis for further detection algorithms tuning

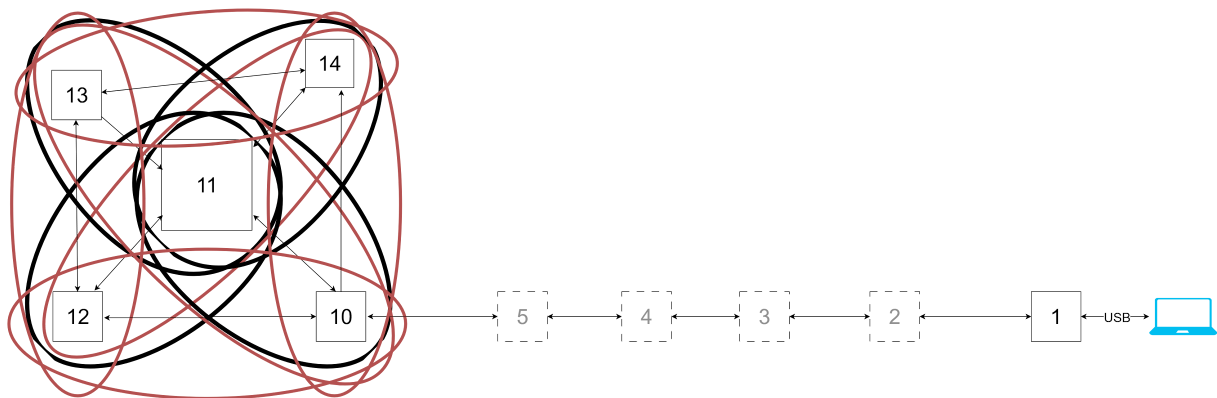
The current demo kits are meant to function as a demonstration and start of integration kit, showing both the detection capabilities together with Mesh communication capabilities. This demo kit is working in standard globally certified unlicensed 2.4GHz band.

Each of the kits consists of 6 main units and 4 extra units:

- 5 detection units - creating possibly detection Ellipse areas of all combinations between them (they also function in a mesh, relaying data through each other)
- 1 Control unit which connects over USB to a control PC
- Optional up to 4 extra Mesh-Relay units to allow further extending the distance between the Control Unit and the detection units, making it a multi-hop Mesh network.

TPL-Signum units are numbered 1,2,3,4,5, 10,11,12,13,14 units. Units 10-14 are detection units. Unit 1 is connected to a terminal through the USB port. Units 2-5 are optional to be provided by TPL, and are potentially meant to be placed between unit 1 and the detection units in order to prolong the distance between the management PC and the detection units. Intruder detection events which are detected between enabled detected units are transmitted directly to unit 1 (if in range) or through units 2-5 (when 1 is not in direct range). Unit 1 then passes the detection information to the management PC through USB over Virtual COM (CDC/TTY) with AT-Commands format, as will be explained throughout this document.

The following is a diagram of the system:



3. Preliminary release of 3 units only

In order to start checking the system, a simplified version is prepared, which is the most basic that allows working with the system.

It includes 3 units:

1. Unit #1 - Management unit which connects to a PC over two USB cables, one for TPL-CLI and one for AT-Commands.
2. Unit #10 - One of the detection units, which generates a detection with Unit #11.
3. Unit #11 - One of the detection units, which generates a detection with Unit #10.

This release is done with firmware version 0.10b219.

4. Installation in the field

Units of the larger set of 20dB demo EVBs are packaged in a basic container to withstand some environmental conditions. They are powered by a battery pack containing 2 AA batteries. To power on and off, the top of the container should be removed by unscrewing four screws, and the switch on the battery pack can be turned on or off.

Units in the 8dBm demo EVBs can optionally be packed in a basic 3D-Printed package. 3D files can be also provided for further editing and printing.

The units can be turned on in any order and will connect and start working.

The 5 detection units, numbered 10-14, are communicating in full mesh, so if unit 11 and 14 are not in range of units 12-13, it can still report on detection through unit 12 and 10.

For the optional units 2-5, if one of the units in the line is not present from the beginning or gets out of sight or turns off, the link jumps over it between adjacent units.

Additionally to the computerized AT-Commands interface in the PC to control the system from a GUI or other management system, it's possible to control the system manually through an additional technical human-friendly textual tool - TPL-CLI. It's possible to use this manual tool to verify everything, as needed.

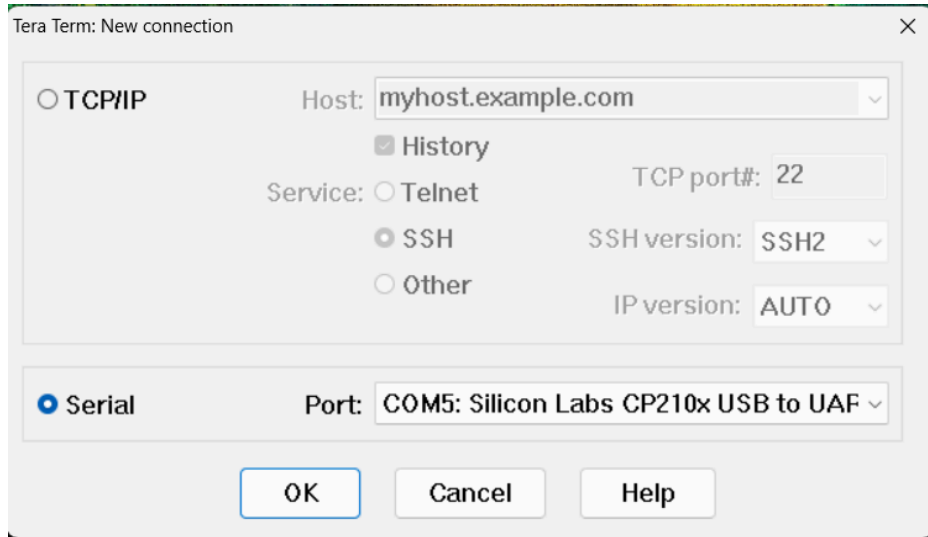
5. TPL-CLI manual human-friendly interface

As said, this tool can be used to help in manual operations, like debugging of the GUI system, system deeper check, or operating functionalities which are not supported by the GUI management system.

This document presents several TPL-CLI commands which can be used, in parallel to the computerized AT-Commands.

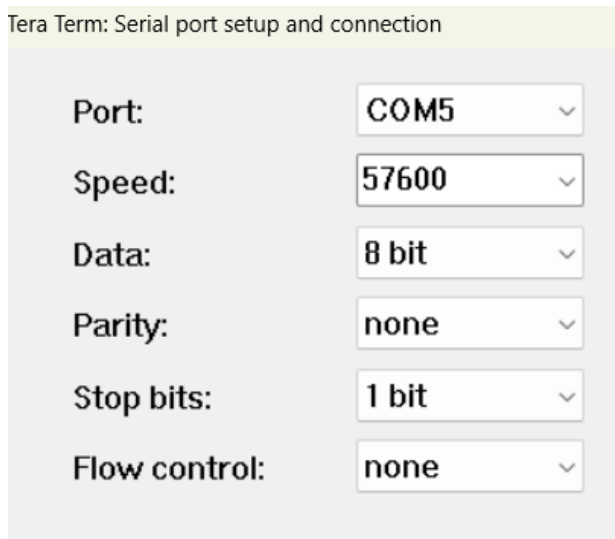
A good tool example on Windows for interfacing with TPL-CLI is TeraTerm:

<https://teratermproject.github.io/index-en.html>



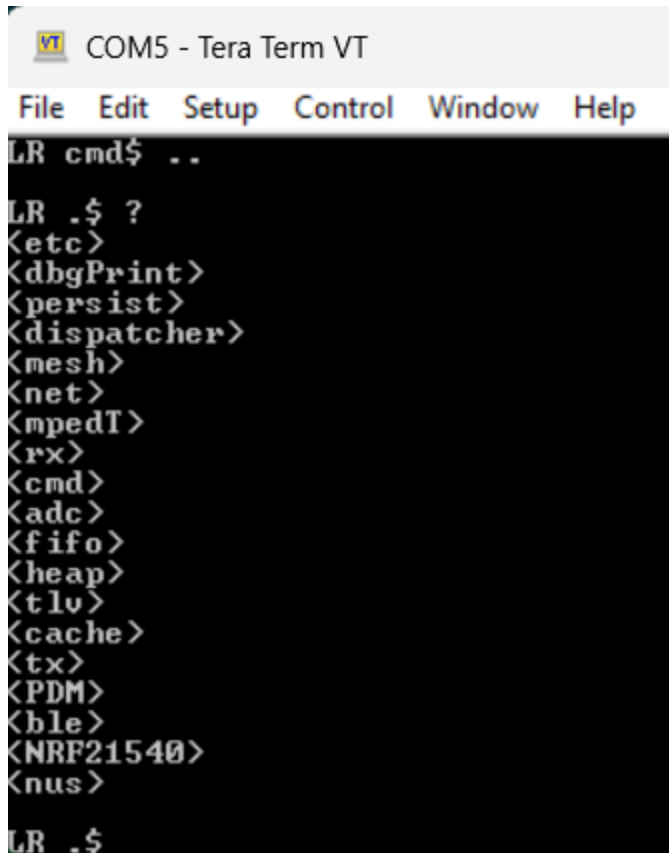
Note, in some setups the naming in the Serial Port is different. You can verify the correct port by unplugging and plugging the needed cable and checking which port appears and disappears in the list while plugging and unplugging.

Configuration for serial connection in TeraTerm:



The TeraTerm tool has multiple useful features which can be used, like logging with Timestamp and Scripting. Scripting capabilities can trigger any other operation through `exec()` command which can trigger Python and other tools to allow event reporting over internet and more.

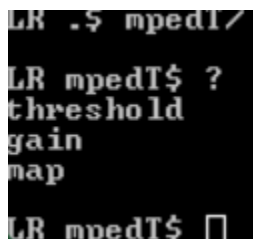
To start working need to enter the `mpedT` folder by typing it with Enter or starting to type with “mp” and using tab key to complete the command:



```

VT COM5 - Tera Term VT
File Edit Setup Control Window Help
LR cmd$ ..
LR .$ ?
<etc>
<dbgPrint>
<persist>
<dispatcher>
<mesh>
<net>
<mpedI>
<rx>
<cmd>
<adc>
<fifo>
<heap>
<tlv>
<cache>
<tx>
<PDM>
<ble>
<NRF21540>
<nus>
LR .$
  
```

Typing “?” with enter will give a list of commands or folders.



```

LR .$ mpedI/
LR mpedI$ ?
threshold
gain
map
LR mpedI$ 
  
```

Key UP is history, which also supports completion using Tab, like in Linux.

6.AT-Commands interface

Interface between Unit 1 and the PC’s GUI management software is performed over “AT-Commands”-based protocol, over serial power (COM/TTY/CDC).

The AT-Commands concept is compliant with standard format of “AT Commands”, as for example described in documents such as “Telit AT Commands Reference Guide”. However, no standard commands are supported, only custom commands for this project are defined.

After completion of every command a value of OK or ERROR is returned. Before either are returned the host should not send any further commands.

All regular command kinds are implemented:

- AT#EXAMPLE=?
 - Test/Help command
- AT#EXAMPLE?
 - Read command
 - Will return values such as:
 - #EXAMPLE=<Result1>,<Result2>
- AT#EXAMPLE=<Value1>,<Value2>,<Value..>
 - Write command
 - Writes the provided parameters in the specific order as defined in each command's format
- AT#EXAMPLE
 - Run command
 - Performs some operation, and it can return some values, similarly to Read command

Each command must complete with CR (Carrier Return), and other end-of-line combinations are also accepted.

All commands will always return OK or ERROR.

Only after issuing the OK or ERROR result from a previous command the system will start analysis of the next command.

Details of the format can be read in spec such as "Telit AT Commands Reference Guide".

7. Intruder Detection Events

Over TPL-CLI

The following is an example of detection events which can be seen on the terminal:

```
[1862125] I CMD:DETECTION ece0(11)-a644(12) th:500 val:555 c:1
[1865126] I CMD:DETECTION ece0(11)-a644(12) th:500 val:542 c:1
[1865201] I CMD:DETECTION a644(12)-ece0(11) th:500 val:551 c:2
[1865726] I CMD:DETECTION ece0(11)-a644(12) th:500 val:709 c:1
[1866746] I CMD:DETECTION ece0(11)-a644(12) th:500 val:635 c:3
```

The most important value in this line is its existence. If such a line is printed, it means that sufficient movement has been detected, and an alert has been triggered.

The *val* field has a significance of indicating a kind of level of detection. It doesn't directly indicate any physical meaning, but in general the higher the value, the higher chance that a real movement has happened.

Display of detection messages to the terminal can be stopped if, for example, they are being displayed so frequently that it is not possible to execute any CLI commands. To stop display of detection messages, enter the *cmd* folder and execute *re* without any parameters.

```
LR .$ cmd
LR cmd$ re
LR cmd$
```

To start display of detection messages, enter *re 3 4*:

```
LR cmd$ re 3 4
```

Over AT-Commands - #EVTDETECT

#EVTDETECT = unit1, unit2, val, th

The *val* field has a significance of indicating a kind of level of detection, as in the event seen on the TPL-CLI terminal.

When the central unit receives a movement above the defined threshold, an “unsolicited” AT CMD message #EVTDETECT is sent. The message contains the numbers of the 2 units between which movement was detected, and parameters describing the detection.

The currently configured threshold is also provided for th.

Suggested GUI display

The EVTDETECT commands are the main commands over AT-Commands interface to allow the PC GUI to receive detection events from the Signum system.

A suggested GUI display method is the following. On App start, it will first display a basic diagram of the hard coded units with the empty ellipses, similar to the above diagrams, with dashed lines instead of solid lines. When movement is detected between any of the units, the ellipse between the units will be shaded. For example, if a person walked between units 11 and 12, the movement would be detected, and the ellipse between the 2 units would be shaded.

val parameter indicates a kind of detection. Values will be from 1 to 1000, to be displayed as gradients in colors between green and blue, linearly interpolated in this range. Values above 1000 should be treated like 1000.

When the GUI application receives an **EVTDETECT** indication, the area defined by the ellipses around the adjacent units should be shaded to indicate that there was movement between the units. Color of the shading is determined by the value of detectionProfile.

Shading should fade away after completion of display of the **EVTDETECT** event.

Gradient example:



TBD (not existing in 0.1b219 release) - Events emulation for PC GUI testing

The central unit contains test software to simulate events and test functions, using a TPL-CLI manual interface. Detection events described above can be simulated using one of the menus of this test software. A single event can be simulated, or multiple events with time delay.

At the prompt, type **SendAT <CR>**.

```

COM9 - Tera Term VT
File Edit Setup Control Window Help
.$ SendAT/
SendAT$ ?
AT      : <atcmd>
EVT      : <srcId> <destId> <Detectionlevel> <certainty> <sensitivity>
EVTPush: <srcId> <destId> <Detectionlevel> <certainty> <sensitivity>
EVTTest: <id1> <id2> <id3> <id4> <numLoops> <delayMs>

SendAT$
SendAT$
SendAT$
SendAT$
SendAT$

```

The **EVTPush** command sends a single event over the BLE UART. **srcId** and **destId** are indexes of the TPL_MESH units.

(EVT command is not important for real operation, used internally by TPL for different mode of events sending. AT command allows sending generic commands over AT-Commands interface, also not relevant for regular operation.)

For example, to send a message simulating an event between units 1 and 2 with detection level 900, certainty 1, sensitivity 8:



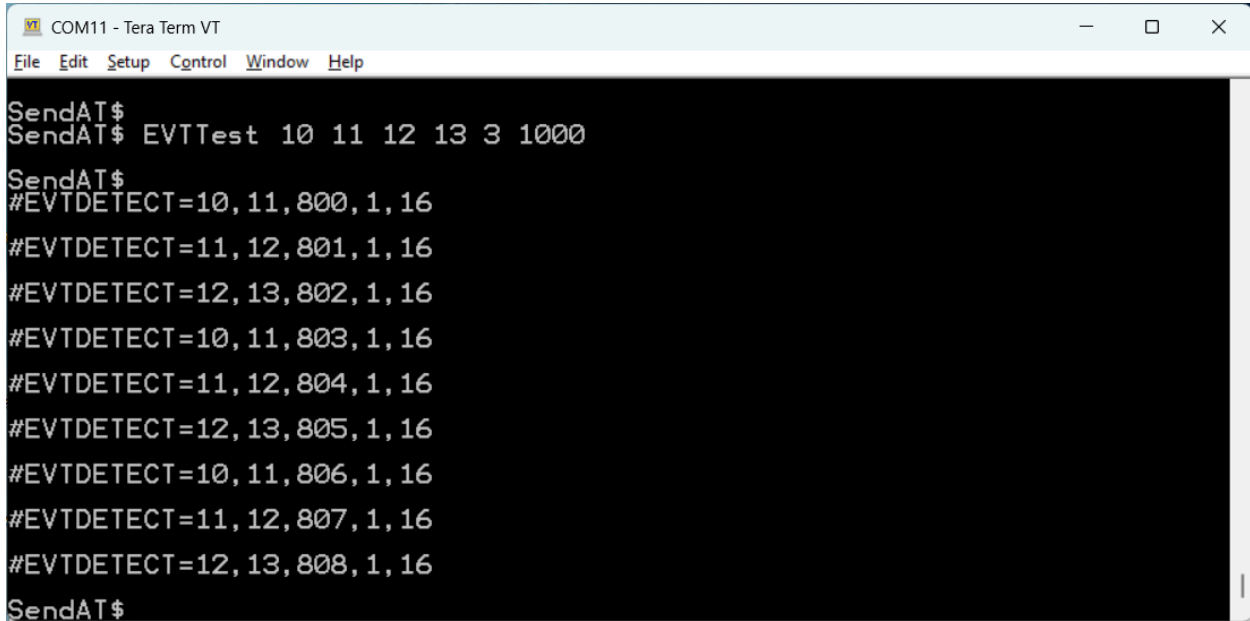
```
COM11 - Tera Term VT
File Edit Setup Control Window Help
SendAT$
SendAT$
SendAT$ EVTPush 1 2 900 1 8
SendAT$
#EVTDETECT=1,2,900,1,8
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
SendAT$
```

The ***EVTTest*** sends a series of events as follows;

DetectionLevel = 800

```
for i=0 - loopCount
    send event id1-id2
    ++DetectionLevel
    send event id2 - id3
    ++DetectionLevel
    send event id3-id4
    ++ DetectionLevel
    Delay delay Ms
```

To send 3 iterations of a series of 3 Detection events, with a delay of 1 second between each series:



```
COM11 - Tera Term VT
File Edit Setup Control Window Help

SendAT$
SendAT$ EVTTest 10 11 12 13 3 1000

SendAT$
#EVTDETECT=10,11,800,1,16
#EVTDETECT=11,12,801,1,16
#EVTDETECT=12,13,802,1,16
#EVTDETECT=10,11,803,1,16
#EVTDETECT=11,12,804,1,16
#EVTDETECT=12,13,805,1,16
#EVTDETECT=10,11,806,1,16
#EVTDETECT=11,12,807,1,16
#EVTDETECT=12,13,808,1,16
SendAT$
```

Detection from communication drop

Over TPL-CLI

The DETECTION-COMM alert indicates that there has been some obstruction between detection units which prevents communication between them. Also if one of the detection units suddenly dies (or gets powered off) this will be the alert. If communication will not resume shortly after, then there will be several such alerts, and eventually a Link-Lost event will get triggered.

```
[58799] I CMD:DETECTION-COMM A644(12)-ECE0(11) 2025
```

```
[58800] I CMD:DETECTION-COMM ECE0(11)-A644(12) 2041
```

Over AT-Commands - #EVTDETCOM

#EVTDETCOM = unit1, unit2, noCommMs, noCommThreshold

Parameters:

- unit1 and unit2 - the IDs of the two units between which the detection has triggered
- noCommMs - The time that has passed since the last successful communication, in milliseconds.
- noCommThreshold - The threshold used to trigger this alert of no communication.

8. Manual Sensitivity Configuration Commands

It's possible to configure some parameters of the current detection algorithm. These parameters will be saved in non-volatile memory, so it will be used in all upcoming runs, including through battery replacements.

Further releases will include additional configuration modes - automatic, semi-automatic, adaptive, etc.

Furthermore, the same Threshold configuration is also used to totally disable detection between two units, and remain only a connectivity link.

Over TPL-CLI

The terminal has a menu system which allows performing multiple operations on the units. The user configuration for this system is located under the mpedT subfolder.

In order to see the list of available folder or commands, you need to type "?" and press Enter.

The following will appear:

```
.$ ?  
<etc>  
.  
.  
<mpedT>
```

In order to get into mpedT folder of commands, it's needed to type in "mpedT" and press Enter. (an alternative is to press "mp", then press Tab for autocompletion, and then press Enter).

The following prompt will appear, indicating that you are now in the correct folder in the menu:

```
mpedT$
```

If you are not able to see or enter into the mpedT menu, then you are probably in a different folder. Type / with enter to get into root folder. Type .. with enter to get into high folders.

To see the available commands in the folder, it's needed to issue "?" command again, to get the following options:

```
mpedT$ ?  
threshold  
gain  
map
```

The current threshold and gain commands allow configuring different parameters for the detection sensitivity.

a. Threshold command

threshold command takes one value as a parameter.

The threshold parameter directly configures the threshold of the last "val" parameter in alert printout, above which the alert will happen and will be printed on the screen.

Example:

LR mpedT\$ threshold 250

[694678] I CMD:th3:250

Decreasing threshold will increase the sensitivity, and thus the frequency of printouts if many of them exist, while increasing threshold will have the opposite effect.

In order to better understand the current environment and detection capabilities it's possible to significantly decrease the threshold parameter, which will result in many prints, allowing the user to learn the system behavior. After learning is completed, and other parameters (i.e. Gain) are configured, the threshold could be set to a sufficiently high level, so that an existence of printout will indicate a real detection event and will prevent false-alarms.

After issuing the Threshold configuration, remember to verify that it was properly received by all units using the map command.

Threshold can be any value between 0 and 65534, Setting Threshold to 0 totally disables detection between the given two units.

b. Gain command

Gain command receives a single parameter and configures the strength of the internal filter. With higher gain values the filtering will be weaker, alerts will be raised closer to the time of the event, but there is also a higher chance for noise and alerts without an actual movement. Again, it's suggested to learn the effects of Gain setting when threshold is configured sufficiently low to see many printed alerts, and in such case Gain can be lowered for stronger filtering which will prevent single high detection events, but could also filter out real movement and add a small delay to the detections.

Example:

mpedT\$ gain 16

Value of 16 is a value that in some cases has been found to be sufficiently strong for outdoor operation and be resistant to some effects like basic wind.

After issuing the Gain configuration, remember to verify that it was properly received by all units using the map command.

Over AT-Commands

Threshold - #SETDETTTHR

AT#SETDETTTHR=<Unit 1>,<Unit 2>, <Threshold>

- <Unit 1>,<Unit 2> - Units to configure the detection between
- <Threshold> - The configured Threshold parameter as described above. 0 totally disables detection.

Gain - #SETDETGAIN

AT#SETDETGAIN=<Unit 1>,<Unit 2>, <Gain>

- <Unit 1>,<Unit 2> - Units to configure the detection between
- <Threshold> - The configured Gain parameter as described above.

9. Configuring detection modes

There are currently several detection modes available, and it's expected in the future that additional modes will get available.

Set Detection Mode - #SETDETMODE

AT#SETDETMODE=<DetectionMode>, <InternalData [HEXBUF]>

- <DetectionMode> - The detection mode to set for the system:
 - 1 - First default supported mode
 - 2 - Second supported detection mode
- <InternalData [HEXBUF]> - Additional internal information for selected mode, as can be provided by TPL for debug, in HEXBUF string format.

OK reply will get generated immediately after receipt of command.

It will take a couple of seconds to issue the request for the new mode, and as the result of that, some devices might get disconnected and connected back again.

It's possible that this command will not actually reach the destination and manage to configure it. First method to check it is to verify that disconnections and back connection really get received.

Second and most recommended method is to verify the configuration using a readback process - #REQDETMODE + #EVTDETMODE.

Request the current detection mode - #REQDETMODE

AT#REQDETMODE

Issues a request from the system to return the current detection mode.

Immediately after the request an OK is returned. Some time afterwards the reply is expected in event #EVTDETMODE.

Some detection modes are configured in remote units, so it will take some time to retrieve the results from those units.

AT#REQDETMODE

OK

Detection Mode Event - #EVTDETMODE

#EVTDETMODE=<DetectionMode>, <InternalData [HEXBUF]>

- <DetectionMode> - The received detection mode of the system
 - a. 1 - First default supported mode
 - b. 2 - Second supported detection mode
- <InternalData [HEXBUF]> - Additional internal information for selected mode, as can be provided for TPL's debug, in HEXBUF string format.

The event is generated following a request to retrieve the information using #REQDETMODE command.

10. Retrieving system status - Map command

Over TPL-CLI

Map command prints information about remote devices configuration and connections between devices.

The commands structure shows which units are in range and connected to which units, including the average RSSI between them.

Additionally, it's possible to read the gain and threshold settings in the set, and it's important to verify that all units in the set have the correct gain and threshold settings configured.

It's also possible to learn the internal identifiers of the units this way to help identify units when multiple sets are in the range, or when stickers fall, etc.

The structure of the commands with interesting parameters explained:

[<timestamp, milliseconds from last powerup/reset, with a wraparound at approximately 49 days>] **I CMD:MAP_RSP from** <identifier of unit which sent this response> **gain:**<gain setting> **voltage:**<reading of battery, when supported> **scan:1 adv1:** [<identifier to first unit connected to the current unit> **th3:**<detection threshold configuration between current unit and this one, and 0 when disabled, that is expected in non-detection connections 1-2,1-11,1-12,2-11,2-12> <average RSSI of communication between these units> **dBm dt:240**] [<Similar structure for second device possibly connected to this unit>] [<Similar structure for third device possibly connected to this unit>]

Example:

LR cmd\$ map

[5044419] I CMD:localidx:1
[5044419] I CMD:idx:1 idx:1

LR cmd\$

[5044913] I CMD:MAP_RSP from 2 ver:0.4c10 gain:64 voltage:2908 scan:1 adv:1: [12 th3:0 -57dBm dt:180] [11 th3:0 -36dBm dt:240] [1 th3:0 -71dBm dt:300]
[5045162] I CMD:MAP_RSP from 11 ver:0.4c10 gain:64 voltage:2925 scan:1 adv:1: [1 th3:0 -78dBm dt:421] [12 th3:500 -52dBm dt:300] [2 th3:0 -39dBm dt:241]
[5045529] I CMD:MAP_RSP from 12 ver:0.4c10 gain:64 voltage:3035 scan:1 adv:1: [11 th3:500 -50dBm dt:300] [2 th3:0 -58dBm dt:180]

We can learn from there the following:

This command was issued from unit 1, and 3 units responded:

- Unit 2 is connected to units 1, 11, and 12. RSSI threshold is 0 between Unit 2 and all other units connected to it, as no detection takes place on Unit 2. Unit 11, with RSSI -36 dbM, is closer than unit 12 with RSSI -57 dbm. Both units are sufficiently close for very stable communication.
- Unit 11 is connected to units 1, 2 and 12. The threshold between unit 11 and unit 12 is 300, as these are the detection units.
- Unit 12 is connected to units 11 and 2. Note that unit 12 is not connected directly to Unit 1, but communicates through units 11 and 2 through the mesh.
- Gain of all units is properly configured to 64.

Over AT-Command

Request Mesh Map - #REQMESHMAP

AT#REQMESHMAP=<Unit or 0 for all>

Requests a specific unit, or all units, in the mesh to show its main status and configuration, including links to surrounding units.

Parameters:

- <Unit or 0 for all> - The specific Unit to ask to send its parameters. Or 0 to ask all units to reply their statuses.

Immediately after sending the request the OK will be returned, confirming the completion of the request transmission.

After that time the replies will start getting received as unsolicited events.

Two different event message kinds will get received, possibly in high amount, depending on the number of devices and links in the mesh:

- #EVTMESHMAPDEV - Single reply from each device in the node
- #EVTMESHMAPDEVLINK - A reply per each link, from one of the sides of the link, while another side will also send another reply for the same bidirectional event.

If interested, it's acceptable to send these AT#REQMESHMAP also again and periodically while there are still some replies getting returned. With such periodic requests and gathering of the replies it's also possible to continuously monitor the mesh state, and all recent RSSI between all units.

Device information event - #EVTMESHMAPDEV

#EVTMESHMAPDEV=<Unit [dec]>,<Version [String]>,<Voltage mV[dec]>,<InternalData [HEXBUF]>

Parameters:

- <Unit [dec]> - The unit ID in the mesh providing the given information
- <Version [String]> - Version of the unit as string. Could contain both firmware and hardware versions.
- <Voltage mV[dec]> - Measured battery voltage of the unit (if supported in specific unit kind), or internal voltage.
- <InternalData [HEXBUF]> - Additional internal information for TPL debug, in HEXBUF string format.

Link information event - #EVTMESHMAPDEVLINK

#EVTMESHMAPDEVLINK=<ReportingUnit>,<LinkedUnit>,<RSSI>,<ThresholdCfg>,<GainCfg>,<InternalData [HEXBUF]>

Parameters:

- <ReportingUnit [dec]> - The unit ID in the mesh providing the given information of this specific link (it's expected that also the second device will report about the same link)
- <LinkedUnit> - The linked device that this device is reporting about
- <RSSI> - Recent RSSI of the link, as measured by the ReportingUnit
- <ThresholdCfg> - Configured Detection Threshold
- <GainCfg> - Configured Detection Gain
- <InternalData [HEXBUF]> - Additional internal information for TPL debug, in HEXBUF string format.

Example

```
AT#REQMESHMAP=0
```

```
OK
```

```
#EVTMESHMAPDEV=10,"SG_0_10b19", 3015,00000000
```

```
#EVTMESHMAPDEVLINK=10,11,-27, 300, 64
```

```
#EVTMESHMAPDEVLINK=10,1,-33, 0, 0
```

```
#EVTMESHMAPDEV=11,"SG_0_10b19", 2926, 00000000
```

```
#EVTMESHMAPDEVLINK=11,1,-41, 0, 0
```

```
#EVTMESHMAPDEVLINK=11,10,-28 ,300,64
```

11. Check communication over mesh - Ping command

Over TPL-CLI

Ping command sends ping request over the mesh to a specific unit or all units.

Response displays device ID and MAC address, and time for the reply to be received.

SG cmd\$ **ping**

```
[1536356] I CMD:PING_RSP from 0x4e1f(10) lat:10386, remLat:-10196, round trip:190
```

```
[1536430] I CMD:PING_RSP from 0x81b0(11) lat:90227, remLat:-89963, round trip:264
```

Lat: time in milliseconds from when the response was sent from the remote unit until it was received by **the** requesting unit. This is valid only if the devices are synchronized

remLat: time in milliseconds from when the request was sent until it was received by the remote unit. This is also is valid only if the devices are synchronized

Round trip: time in milliseconds from when the request was sent by the requesting unit until it was received by the requesting unit. This does not require synchronization between devices.

Over AT-Command

PING Command transmission over the network - #PING

AT#PING=<Unit or 0 for all>

Sends ping request to a specific unit or all units.

Parameters:

- <Unit or 0 for all> - The specific Unit to which to send ping request, or 0 for all units

Immediately after sending the request the OK will be returned, confirming the completion of the request transmission.

After that time the replies will be received as unsolicited events.

In the response to the AT command, only the round trip time is displayed.

Example

AT#PING = 0

OK

#PINGRSP=10,160

#PINGRSP=11,232

Indicates that the response was received from unit 10 160 msec after it was requested, and the response from unit 11 was received 232 msec after it was requested.

12. Link-Up/Link-Down events

Over TPL-CLI

If a connection (Link-Up) or disconnection (Link-Down) occurs between one or more devices, a message will be printed to the terminal.

The following is an example of a disconnection message, where unit 2 and unit 11 have lost connection. Disconnection messages often follow a series of DETECTION-COMM events. The 0 at the end of the line indicates disconnection and in following case unit 2 reports that it got disconnected from unit 11:

```
[1531538] I CMD:CONNECTED 376F(2) connected:ECE0(11) 0
```

The following is an example of a message of connection between unit 1 and unit 2. The 1 at the end of the line indicates connection:

```
[1531660] I CMD:CONNECTED EDC5(1) connected:376F(2) 1
```

Over AT-Commands

Link established event - #EVTMESHLINKUP

#EVTMESHLINKUP=<ReportingUnit>,<LinkedUnit>,<RSSI>,<ThresholdCfg>,<GainCfg>,<InternalData [HEXBUF]>

Example:

```
#EVTMESHLINKUP= 11,1,0,0,0
```

```
#EVTMESHLINKUP=11,10,0,250,64
```

```
#EVTMESHLINKUP=10,11,0,250,64
```

Parameters:

- <ReportingUnit [dec]> - The unit ID in the mesh providing the given information of this specific link (it's expected that also the second device will report about the same link)
- <LinkedUnit> - The linked device that this device is reporting about
- <RSSI> - Recent RSSI of the link, as measured by the ReportingUnit, might be 0 if the RSSI is not known yet.
- <ThresholdCfg> - Configured Detection Threshold
- <GainCfg> - Configured Detection Gain
- <InternalData [HEXBUF]> - Additional internal information for TPL debug, in HEXBUF string format.

Example:

Link dropped event - #EVTMESHLINKDOWN

#EVTMESHLINKDOWN=<ReportingUnit>,<PreviouslyLinkedUnit>,<LastRSSI>,<Reason>

Parameters:

- <ReportingUnit [dec]> - The unit ID in the mesh providing the given information of this specific link (it's expected that also the second device will report about the same link)
- <PreviouslyLinkedUnit> - The linked device that this device is reporting about
- <LastRSSI> - Last RSSI of successful transfer before the link dropped, as measured by the ReportingUnit.
- <Reason> - The reason why a device got disconnected from the second device, supported values:
 - TBD - Timeout, i.e. unexpectedly packets stopped arriving
 - TBD - Locally initiated drop, i.e. some algorithm of the local device has initiated the disconnection
 - TBD - Remotely initiated drop, i.e. some algorithm of the linked device has initiated the disconnection
 - TBD - TBD

If a link on a detection unit drops, a series of EVTDETCOM messages will be displayed, with the interval increasing on each line, followed by an EVTMESHLINKDOWN message.

Note: As of version 0.1b219 there is an issue that disconnection even doesn't appear over AT-Commands when the disconnection is from the local, central device.

Example:

EVTDETCOM= 10, 11 6866

.
.
.

EVTDETCOM= 10,11, 27867

EVTMESHLINKDOWN = 10,11,0,8

13. Antenna selection

On the units of Set 1 and Set 3, it's possible to select the used antenna. The units include an on-board antenna which is great for isotropic (3D all directions) transmission, and it's possible to attach external antennas on the UFL connector.

Set selection of current active antenna - #SETACTANT

AT#SETACTANT=<Unit or 0 for all>,<ActiveAntenna>

- <ActiveAntenna> - Units to configure the detection between
 - 1 - Internal isotropic antenna
 - 2 - External antenna

OK reply will get generated immediately after receipt of command.

(TBD - ERROR will get replied if the requested unit doesn't support the requested kind of antenna.)

It will take a couple of seconds to issue the request for the new mode, and as the result of that, some devices might get disconnected and connected back again.

It's possible that this command will not actually reach the destination and manage to configure it.

The recommended method to verify the configuration using Read operation - AT#REQACTANT.

Request current selected active antenna - #REQACTANT

AT#REQACTANT=<Unit or for all>

- <Unit> - The unit to request for antenna selection from, or 0 to request the reply from all units.

As a result #EVTACTANT will be replied from the requested unit, or from all units when requested with 0.

OK is returned immediately.

Example to request antenna configurations from all the units:

AT#GETACTANT=0

OK

Selected active antenna report event - #EVTACTANT

#EVTACTANT=<Unit>,<ActiveAntenna>, <SupportedAntennas>

- <Unit> - The unit reporting the antenna selection.

- <ActiveAntenna> - Units to configure the detection between:
 - 1 - Internal isotropic antenna
 - 2 - External antenna
- <SupportedAntennas>
 - 1 - Only internal isotropic antenna supported
 - 2 - Only external antenna supported
 - 3 - Both internal isotropic and external antennas supported.

Replied event following a request from #GETACTANT command.

14. DFU - Firmware update through phone

The update is performed using nRF Connect for Mobile app on Android or iOS.

nRF Connect for Mobile

1. Entering DFU mode:

Over TPL-CLI

To update units other than unit 1, in TPL_CLI cmd menu execute:

```
cmd$ enterBootloader -i xx
```

where xx is the Unit number of the board to be updated (2,11, or 21) After the command is executed, board xx will enter DFU mode.

To update unit 1, exit the cmd folder and enter the etc folder, then execute:

```
LR etc$ enterBootLoader
```

After pressing enter , the following will be printed to the terminal:

```
ENTERING BOOTLOADER
```

After the command is executed the board will enter DFU mode, so remote commands to the device (like map) and mesh communication through it will not work, and location connection to the terminal (if attached) won't work either.

2. Connecting over BLE

Run nRF Connect for mobile phone. Enter dfu as filter

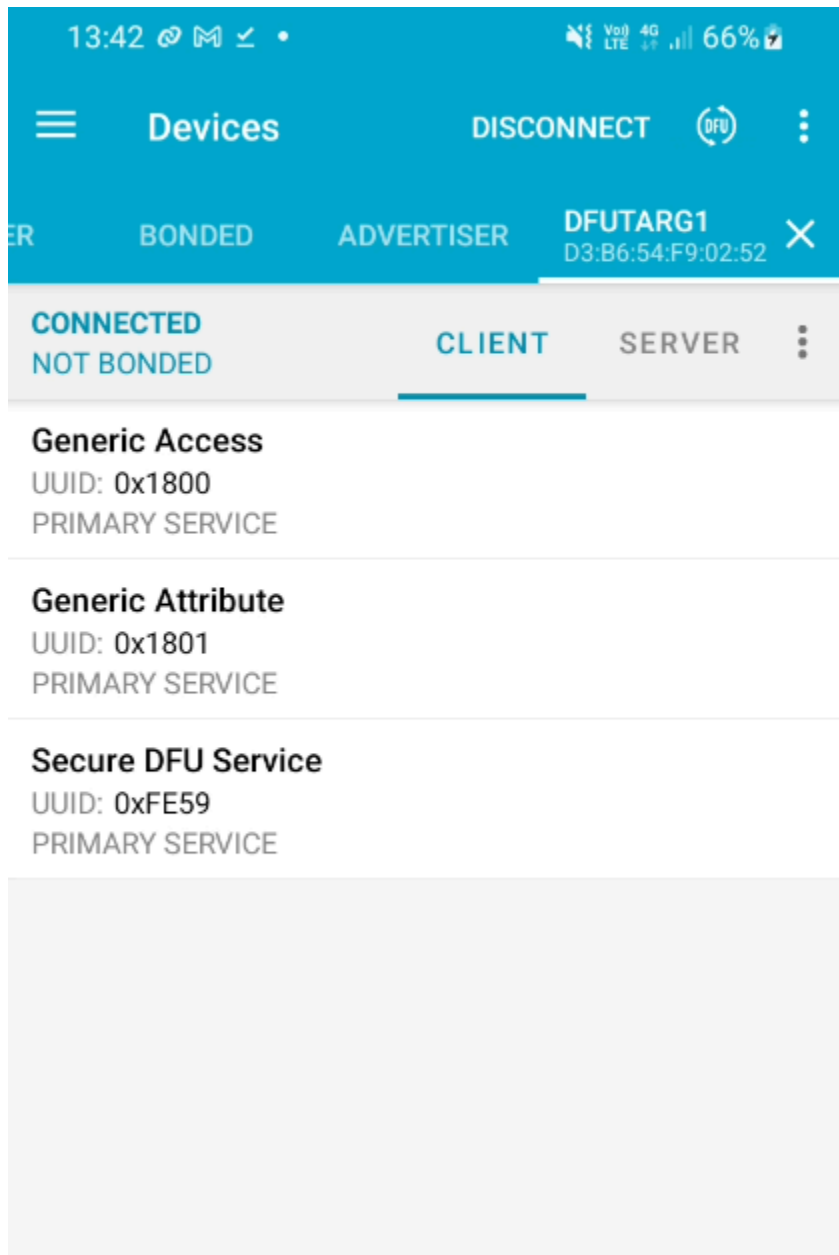
Start scan. Bootloader will show up as device DfuTarg1

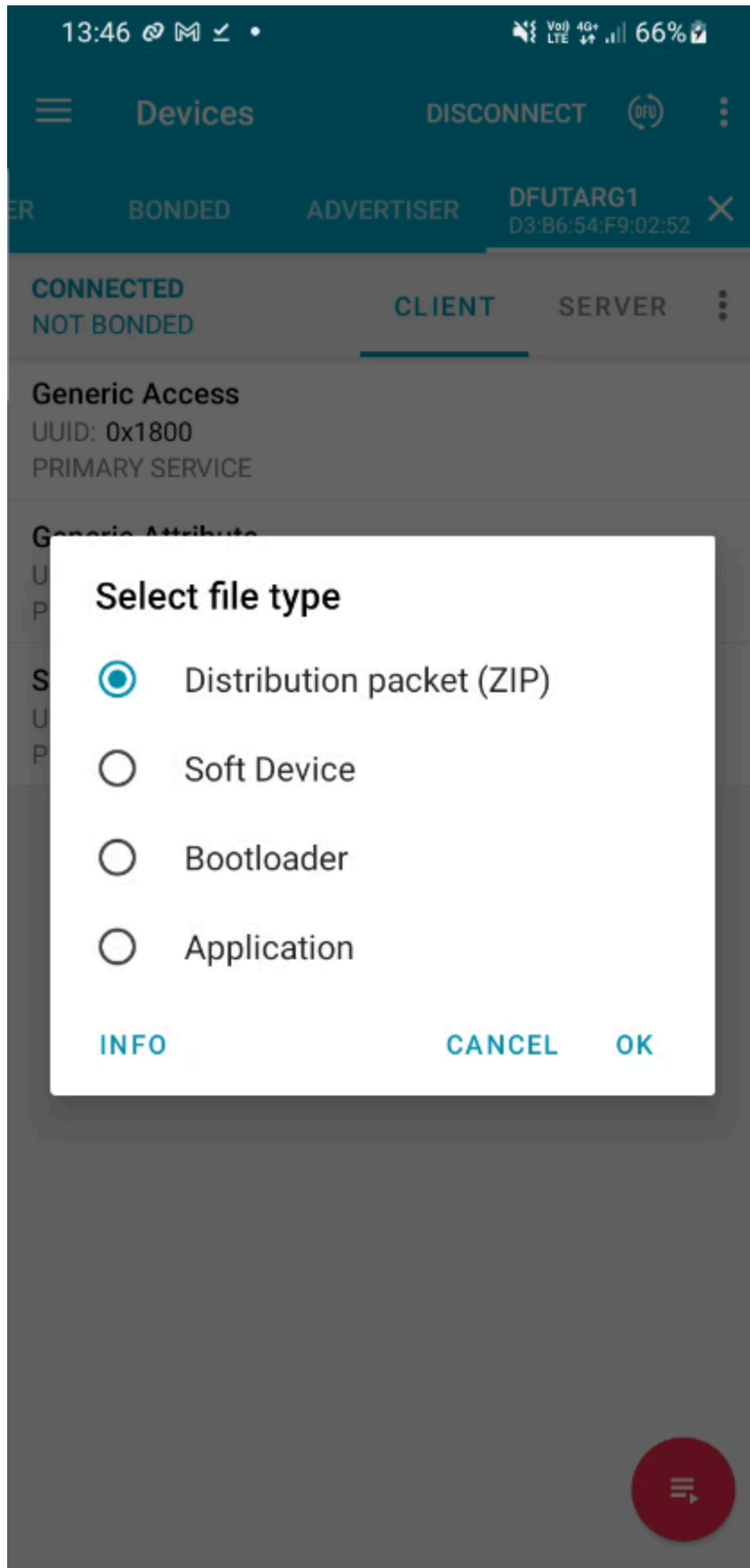
Connect to DfuTarg1 :

If connection is not made and transfer started within 2 minutes after entering bootloader mode, the bootloader will exit bootloader mode and load the existing application (process will need to be restarted to re-enter bootloader again).

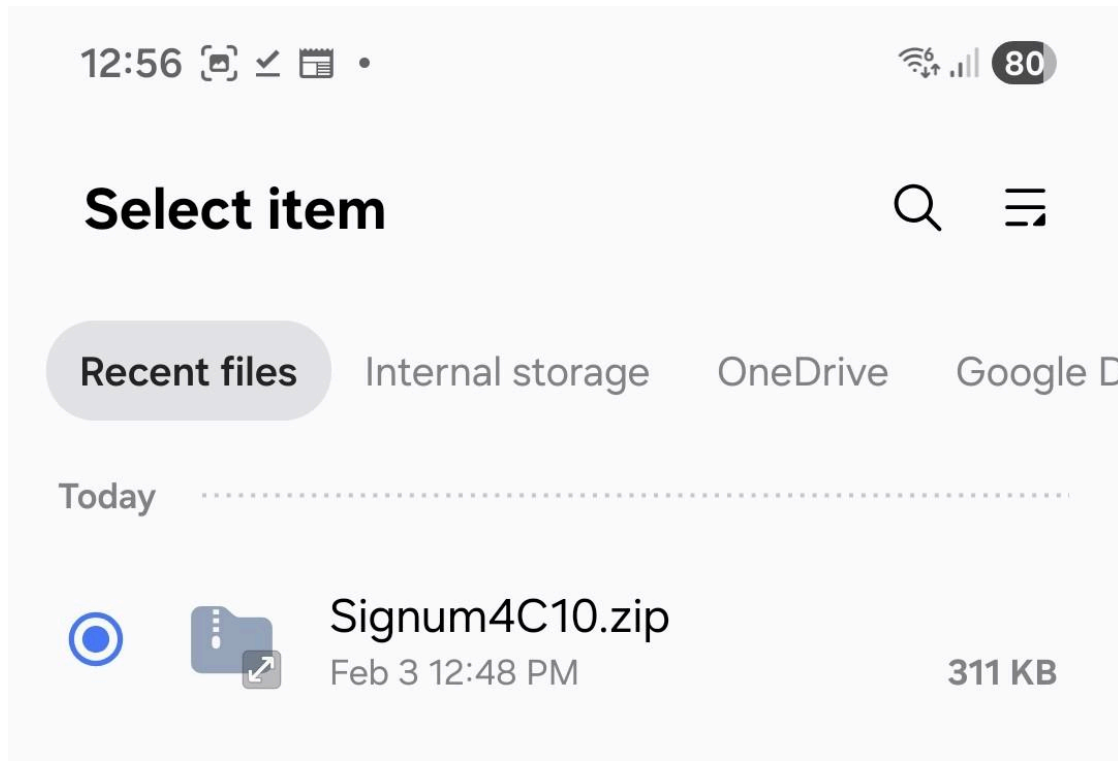
3. Transferring application file

On the DfuTarg1, the DFU icon (to the right of DISCONNECT) will start secure DFU. Click the icon and a window for choosing zip file to transfer will appear:

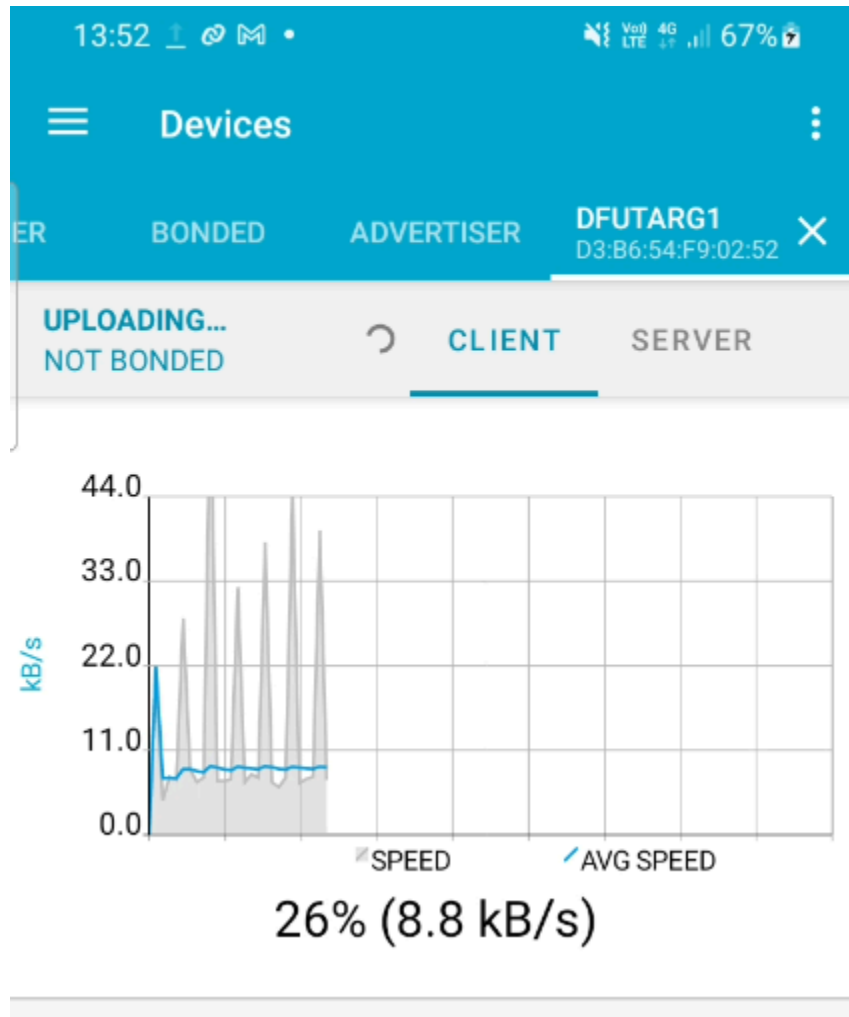




Choose .zip file The file name will change depending on the board type and version number:



Click on **Start DFU** and transfer will start.



The transfer takes 1-2 minutes. On completion, the updated application has been loaded and the board will start advertising as mesh xx

The updated application has been loaded and should be running.

15. Pure mesh commands - not fully operational in this released firmware

Mesh Commands

This section includes commands that are related to Mesh communication, these commands affect the events depicted in the next subchapter (Mesh Events)

Send message – #MESHSEND

AT#MESHSEND=<Destination ID[2 bytes]>,<User Message>

The User Message will be of TBD length and will also be in hexadecimal format.

The TTL level and HOLDOUT_TIME are predefined by the settings commands.

Will send a message to the destination device over the Mesh network.

The network will make the device, or multiple devices, which have the Destination ID in their own ID lists, to receive this message.

This function is not meant to configure the routing methods.

Meaning, by default, any device with a different ID than the destination ID the message is intended to, will forward the message, but will not present it to its local host controller. (There might be separate methods to configure the routing methods to optimize the network performance.)

If there is any detected error with the transmission, the value ERROR will be returned.

On a presumably successful sending, the value OK is returned.

Mesh Events

This section includes the unsolicited events that are triggered from Mesh operations and coexists with the former subchapter (Mesh Commands)

Mesh received data event - #EVTMESHRCV

#EVTMESHRCV=<Source ID[2 bytes]>,<Destination ID[2 bytes]>,<User Message>,<Net Source[2 bytes]>,<TTL_left[1 byte]>

Message event received from the Mesh network.

The Destination ID is the ID to which this message was sent to, out of the configured RCVIDs. Same message might get received several times, or not received at all. Also low chance, but possible that some messages will arrive corrupted. So recommended that the application layer will have its own message integrity checks and possible retransmits.

Mesh Settings commands

These commands will change the network performance and operation.
Once set, all related ble commands will act accordingly.
Can be changed differently during operation (between send commands)

Create Network ID and Security - #CREATENETSEC

AT#CREATENETSEC=<Handle [dec]>

Will create a new supported Network with its Security kind configuration.

The actual security parameters and keys are set using different options, like full provisioning flow, or hard-coded keys with dedicated commands.

<Handle> - A decimal value to identify this specific configuration for further operations with them.

Get Network ID list - #GETNETSEC

TBD

Select Current Network ID and Security for TX - #SETTXNETSEC

AT#SETTXNETSEC=<Handle [dec]>

Will configure the network ID (and appropriate security) to be used for sending the next data (like with #MESHSEND).

Example of usage for scenario of a large network which needs to be divided to smaller networks for enhanced performance, each with dedicated gateway, and a single trivial security for the whole network to be separated from other networks with simple encryption:

Create a the first network and security for highest level, including the most trivial hex encryption. Id assigned for this special kind is 1. There are no upper nodes, so parent is 0. The Network ID for this large network is 1001 which is the current ID allocated to its dedicated area.

AT#CREATENETSEC=1,0,1001,1

Create inner (lower configuration), just under the encryption level, and specifies that in this layer, with address 51 marking that this is part of small network around Gateway number 51.

AT#CREATENETSEC=10,1,51,0

Another network near another gateway, called 52:

AT#CREATENETSEC=11,1,52,0

Will mark that transmitted packets will go only to the local gateway.

AT#SETTXNETSEC=10

For initial versions, and for backward compatibility, devices that will not give this command will send everything in a non network supported format. (meaning in the parent root 1 network)

After setting the TXNETSEC once, all future commands sent will be in network format and will not be received by older versions which do not support subnets. (even if not encrypted)

To return to send in the non network mode, set the Handle to the root 0 :

AT#SETTXNETSEC=0

Delete Network ID and Security - #DELNETSEC

AT#DELNETSEC=<Handle [dec]>

- Will delete the network formerly created by #CREATENETSEC and no longer receive messages from it (leave network)
- Will return error in the following cases:
 - Network handle do not exist
 - Network contains subnetworks (defined as parent handle for another net)

Set VNet Virtual Network - #SETVNET

AT#SETVNET=<VNet value[1 byte]>

0-254: Values of virtual network which can cross multiple networks and still communicate with the same Source/Destination IDs.

255: Special value of 255 marks this Virtual net to be exactly the same as the current network. (Another thought was to use the special value as MSB only, and allow further 7 bits to identify something, but the reason what to identify wasn't found yet.)

When not configured, a default of 0 is preconfigured. Meaning that if all devices have the same VNet of 0, then all NetID and RcvID values are global to the whole system.

Set mesh device ID - #SETID

AT#SETID=<Device ID[2 byte]>

Will set the device ID per unit for transmitted messages.

This is a must for any transmission. If it's not configured a transmission request will return an ERROR.

This is a single (and usually unique) ID per unit.

By default, an ID of 0 is configured after boot.

Add mesh device ID for receiving - #ADDRRCVID

AT#ADDRRCVID=<Device ID[2 byte]>

Will set the device ID per unit for received messages.

A number of ID's can be assigned per single unit. Several #ADDRRCVID invocations will add several IDs per unit.

Get mesh device IDs for receiving - #GETRCVID

AT#GETRCVID?

Will return a list of all the device IDs that have been configured.

Example:

AT#ADDRRCVID=10

OK

AT#ADDRRCVID=11

OK

AT#GETRCVID?

#GETRCVID: 10

#GETRCVID: 11

OK

Delete mesh device ID for receiving - #DELRCVID

AT#DELRCVID=<Device ID[2 byte]>
Will delete a device ID formerly set by #ADDRVID

Example:

```
AT#ADDRVID=10
OK
AT#ADDRVID=11
OK
AT#GETRCVID?
#GETRCVID: 10
#GETRCVID: 11
AT#DELRCVID=10
AT#GETRCVID?
#GETRCVID: 11
```

Set TTL level - #SETTTL

AT#SETTTL=<TTL value[1 byte]>

Will set the TTL for any new outgoing messages from this device to the specified value given.
Can be called during normal operation and between messages.
Will return OK
Possible TTL value: 1 - ff

Example:

```
AT#SETTTL= 2
OK
AT#MESHSEND=1,1abf
OK
AT#SETTTL= ff
OK
AT#MESHSEND=2,125467af
OK
```

This will send a message for a short range to device with ID=1 with the value of 0x1abf, And after will send a long range message with the value 0x125467af to device ID=2.

The default TTL after boot is TBD.

Set Holdout Time level - #SETHOLDOUTTIME

AT#SETHOLDOUTTIME=<HOLDOUT_TIME value[DEC]>

Sets the time that outgoing messages are withheld and continue to try being sent before they timeout and get deleted.

Specified value given in steps of 100ms.

W

Can be called during normal operation and between messages.

Will return OK

Possible HOLDOUT_TIME values: 1 - 65535

Example:

```
AT#SETHOLDOUTTIME= 2
```

```
OK
```

```
AT#MESHSEND=1,1abf
```

```
OK
```

```
AT#SETHOLDOUTTIME=240
```

```
OK
```

```
AT#MESHSEND=2,125467af
```

```
OK
```

This will send a message for a short duration of 200ms to device with ID=1 with the value of 0x1abf, And afterwards will send a longer duration of 25.5 seconds message with the value 0x125467af to device ID=2.

Important to remember, unless otherwise noted, each device forwarding this message will start new timer and send the message with a duration of HOLDOUT_TIME it has defined, and not the original sender's HOLDOUT_TIME .

Set Switches Per Message (internal) - #SETTSWPERMSG

```
AT#SETTSWPERMSG=?
```

```
#SETTSWPERMSG=<SW_COUNT[DEC]>
```

This is an internal parameter.

Example:

```
AT#SETTSWPERMSG?
```

```
#SETTSWPERMSG=50
```

```
OK
```

```
AT#SETTSWPERMSG=45
```

```
OK
```

Set Counts per Switch (internal) - #SETCNTPERSWITCH

```
AT#SETCNTPERSWITCH=?  
#SETCNTPERSWITCH=<SETCNTPERSWITCH[DEC]>
```

This is an internal parameter.

The default after boot is 50ms.

Example of read command when the configuration is set to 50ms (like the default state):

```
AT#SETCNTPERSWITCH?  
#SETCNTPERSWITCH=50  
OK
```

Set Dynamic Params (internal) - #DYNSETPARAMS

```
AT#DYNSETPARAMS=<Interval1>,<Interval2>,<Window2>
```

These are internal parameters.

When not configured, default values are used.

Units of each are TBD.

```
AT#DYNSETPARAMS?
```

```
#DYNSETPARAMS=64,80,48
```

```
OK
```

```
AT#DYNSETPARAMS=70,80,55
```

```
OK
```

```
AT#DYNSETPARAMS?
```

```
#DYNSETPARAMS=64,80,55
```

Set priority #SETPRIORITY

```
AT#SETPRIORITY=<Priority>
```

TBD

Save current settings #SAVESETTINGS

#SAVESETTINGS=<Settings slot[1 byte]>

Will save the current settings to a slot with the given number.

Load settings #LOADSETTINGS

#LOADSETTINGS=<Settings slot[1 byte]>

Will load the settings from the given slot to the current settings the device will act according to.
If an empty slot will try to be loaded, the function will print ERROR.

Delete settings #DELSETTINGS

#DELSETTINGS=<Settings slot[1 byte]>

Will Delete the settings on the given slot.

Print settings list #PRTSETTINGS

#PRTSETTINGS?

Will print a list of all the settings saved to different slots.
This in fact will print the slot following with the result of each get function of each of the settings.

<slot#>,<setting 1 name>=<setting 1 value>,<setting 2 name>=<setting 2 value>,...,<setting n name>=<setting n value>

Example:

AT#PRTSETTINGS?

#PRTSETTINGS: 1,TTL=1,HOLDOUT_TIME=100,PRIORITY=5

#PRTSETTINGS: 2,TTL=20,HOLDOUT_TIME=10,PRIORITY=9

OK

Management Commands

Init (not necessary) - #INIT

AT#INIT

The command is planned to be supported, but doesn't do anything, and is not necessary to issue.

It could be used as a placeholder for future initialization operations.

Reply:

OK

Get version - #GETVERSION

AT#GETVERSION?

Get firmware version of the project.

As a result, the firmware version will be returned.

Returned version format:

OK

#GETVERSION:<firmware version>

For example:

firmware version in 1.0b24 :

AT#GETVERSION=?

OK

#GETVERSION:1.0b24

Reset the device - #RESET

Reset the device.

Notice that in current historical implementation the returned print of “ATCMD_CLI_READY” indicates that a reset has happened. Reset might also happen in other occasions (like Watchdog event), and it can be detected by continuously monitoring the ATCMD_CLI_READY. An improved method for ramp-up after reset is upcoming.

AT#RESET

Returned version format:

ATCMD_CLI_READY

AT#RESET?

Returned version format:

OK

Set print level - #SETPRTLVL

AT#SETPRTLVL=<level>

Will enable or disable the #ERROR and #TRACE events according to given level.

Level parameter:

1. Enable all
2. Disable #TRACE event
3. Disable #TRACE and #ERROR events

Start cli uart - #TPLCLIUARTSTART

AT#TPLCLIUARTSTART

Started the CLI menu in another COM, Returned version format:

In the other COM <TPL_CLI_READY>

Will return OK at the current COM.

Start cli uart - #TPLCLISTART

AT#TPLCLISTART

Show all AT_CMD commands- #HELP

AT#HELP

Present all the optional commands

Get mac address device - #GETMAC

AT#GETMAC=?

#GETMAC=<MAC[HEX]>

TBD prints local device mac address.

OK

Will print the Mac address's board

AT#GETMAC?

#GETMAC=EBF51219EB9B

OK

Echo the atcmd- #ECHO

AT#ECHO=?

For example:

#ECHO=<ECHO:UINT32[RW]>

Will return 0 to disable atcmd echo, 1 to enable

Configure Baudrate - #BAUD

AT#BAUD=?

#BAUD=<baudrate[DEC]>

Set UART baudrate. The OK is returned already in the new baudrate.

Supported values are: 115200, 57600 and 9600.

AT#BAUD?

#BAUD=0

OK

Management Events

Device Error event - #EVTERR

#EVTERR: <error number>,<Error text>

This event will act as a system error indication.

Upon known system errors or conditions that the device should not be in during normal operation, this event will be issued.

The error number is TBD and will define the type of error.

Trace print event - #EVTTRACE

#EVTTRACE: <text>

This event will act as a debug feature, during development, trace level print events will be inserted in selected points in the code, for production, this print can be configured off.

Commands for Firmware Update (DFU) over AT-Commands

Prepare For Update - #PREPDFU

AT#PREPDFU=<Update Type>,<Update Id[dec]>,<Package Length [Bytes]>

The command should be invoked before starting to download the image. This will check if there is enough space for the image, and will prepare the space by erasing it.

Please notice that erasing process might take a long time, so the completion event will get received using asynchronous event #EVTPREPDFUCOMPLETE.

Update allows support for different future update types. The first default update will have the value 0x01.

#PREPDFU will always return OK (unless command doesn't exist, or something critical like that)

Update Type:

1 - default DFU

Update Id: specific id for each update

Prepare for update complete unsolicited event - #EVTREPDFUCOMplete

#EVTREPDFUCOMplete=<Update Id[dec]>,<Status>

Status:

0 - Success

1 - Fail, insufficient space on internal flash, and external flash not supported (first to be implemented)

2 - Fail, insufficient space on internal flash, and external flash missing

3 - Fail, insufficient space on internal flash, nor on external flash

4 - Fail, insufficient space on external flash (internal flash storage not supported)

5 - Fail, busy (when another request is sent, and previous request is still executing)

6 - Fail, unknown

Write DFU Data - #WRITEDFUDATA

AT#WRITEDFUDATA=<Update Id[dec]>,<offset[dec]>,<length[dec]>,<data[hex]>

Before issuing this command the DFU update should be prepared using a call to #PREPDFU. If the #PREPDFU will not get called, there is a risk that this command will fail, while returning ERROR (if the space is not erased, or we don't have a stored start address of the package, possibly due to reset in between.).

The data must be of the same size as the <length> parameter (counting binary buffer bytes), otherwise the function will fail.

Update Id should be the same as the same as provided in #PREPDFU. Different one will generate an error. First implementations will have this stored in RAM, which will force full image downloading in a single session without resets in between.

Note: In Nordic nRF52 implementation, it's impossible to write to the same 32bit word more than twice between erases (between calls to #PREPDFU).

If any error will occur the result will be:

ERROR

If it's interesting to get more information about the failures, then it's needed to use the asynchronous version of this command - #WRITEDFUDATAASYNC.

If all succeeded, it will return:

OK

Example:

AT#WRITEDFUDATA=1,192,64,0102030405060708090A0B0C0D0E0F1011121314151617181
91A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3
D3E3F40
OK

Write DFU Data - #WRITEDFUDATAASYNC

AT#WRITEDFUDATAASYNC=<Update Id[dec]>,<offset[dec]>,<length[dec]>,<data[hex]>

The data must be of the same size as the <length> parameter (counting binary buffer bytes), otherwise the function will fail.

Upon the completion of the write operation into the flash (might be long when radio in Nordic is busy), then a separate event will get generated: #EVTWRITEDFUDATA

Depending on final implementation, several such requests could be enqueued in parallel, while previous requests are still being handled. These however surely will not be large to support full images, as it will require a very large RAM buffering. Maybe 1 additional parallel request will be supported to allow quick back-to-back write operations, without the idle time to wait for a new request to be generated from the host after receiving the previous events.

#WRITEDFUDATAASYNC will always return OK (unless command doesn't exist, or something critical like that)

Write DFU Data complete event - #EVTWRITEDFUDATA

#EVTWRITEDFUDATA=<Update Id[dec]>,<offset[dec]>,<Status>

Status:

0 - Success

1 - Fail, requested to write out of prepared area

2 - Fail, failure to write, probably not erased (or dead flash after about 10k number of writes)

3 - Fail, invalid <Update Id>. There was either a reset after calling PREPDFU, or PREPDFU wasn't called before the call to EVTWRITEDFUDATA.

4 - Busy. Can't enqueue additional parallel asynchronous write requests.

5 - Fail, unknown

Offset: offset of the sent packet for identification

Read DFU Data - #READDFUDATA

AT#READDFUDATA=<Update Id[dec]>,<offset[dec]>,<length[dec]>

#READDFUDATA:<Update Id>,<Status>,<offset[dec]>,<length[dec]>,<data[hex]>

OK

Status:

0 - Success

1 - read failed - offset out of range - data value will give 0

2 - read warning - length exceeded max package address - can be used to indicate last package read. Data will give data only up to the last valid package address.

3 - Fail, invalid <Update Id>. There was either a reset after calling PREPDFU, or PREPDFU wasn't called before the call to EVTWRITEDFUDATA.

Read DFU Data - #READDFUDATAASYNC

AT#READDFUDATAASYNC=<Update Id>,<offset>,<length>

OK

..

..

#EVTREADDFUDATA:<Update Id>,<Status>,<offset>,<length>,<data>

Depending on final implementation, several such requests could be enqueued in parallel, before previous requests are still being handled. These however probably will not be very large. Maybe 1 additional parallel request will be supported to allow quick back-to-back write operations, without the idle time to wait for a new request to be generated from the host after receiving the previous events.

AT#READDFUDATAASYNC will always return OK (unless command doesn't exist, or something critical like that)

Read DFU Data complete event - #EVTREADDFUDATA

#EVTREADDFUDATA:<Update Id>,<Status>,<offset>,<length>,<data>

Status:

0 - Success

1 - read failed - offset out of range - data value will give 0

2 - read warning - length exceeded max package address - can be used to indicate last package read. Data will give data only up to the last valid package address.

3 - Fail, invalid <Update Id>. There was either a reset after calling PREPDFU, or PREPDFU wasn't called before the call to EVTWRITEDFUDATA.

4 - Busy. Can't enqueue additional parallel asynchronous read requests.

5 - Fail, unknown

Offset: relative address that read data start from

Length: number of byte that actually read

Check Update package - #CHKDFUPKG

AT#CHKDFUPKG=<Update type>,<Update Id>,<Check Type>

OK

#EVTCHKDFUPKG:<Update Id>,<Status>,<Check Type>

This is an asynchronous command meant to check if the data that was written to the flash is full and valid.

*this command is not required during DFU since the check operation is being performed anyway during #EXEDFU.

Update Type:

1 - default DFU

Check Type:

1 - Package wrapper

- 2 - Check package data (in addition to wrapper), might take longer time
- 3 - Dry run with resets (will not be supported in first versions)

Check Update package complete event - #EVTCHKDFUPKG

#EVTCHKDFUPKG:<Update Id>,<Status>,<Check Type>

Status:

- 1 - Success from #EXEDFU
- 2 - Success from #CHKDFUPKG
- 3 - Fail, invalid <Update Id>. There was either a reset after calling PREPDFU, or PREPDFU wasn't called before the call to EVTWRITEDFUDATA.
- 4 - Data check failed
- 5 - Package check failed
- ~~5 - Version incompatibility - current version is higher then new version~~
- 6 - Failed to update bootloader settings
- 7 - unknown error

Execute DFU - #EXEDFU

AT#EXEDFU=<Update type>,<Update Id>

OK

#EVTCHKDFUPKG=<Update Id>,1

(New version upload)

Update Id - In first versions will not compare with Update Id of the actual downloaded chunks. This will allow delayed installation (with check) anyway without this protection.

This command will start to update the current application to the new version currently loaded. Prior to the update a full package check will be performed (wrapper and data, same as in #CHKDFUPKG) and will return a #EVTCHKDFUPKG event as well.

Upon a successful check complete event the update itself will begin (there will be no other indication to start of loading).

In future releases, there will be a progress indication prints in the form of (possibly from the bootloader, or other participating parties):

#EXEDFU: <Msg id[int]>, <Progress[string]>

Msg Id [int] - integer value representing the current message

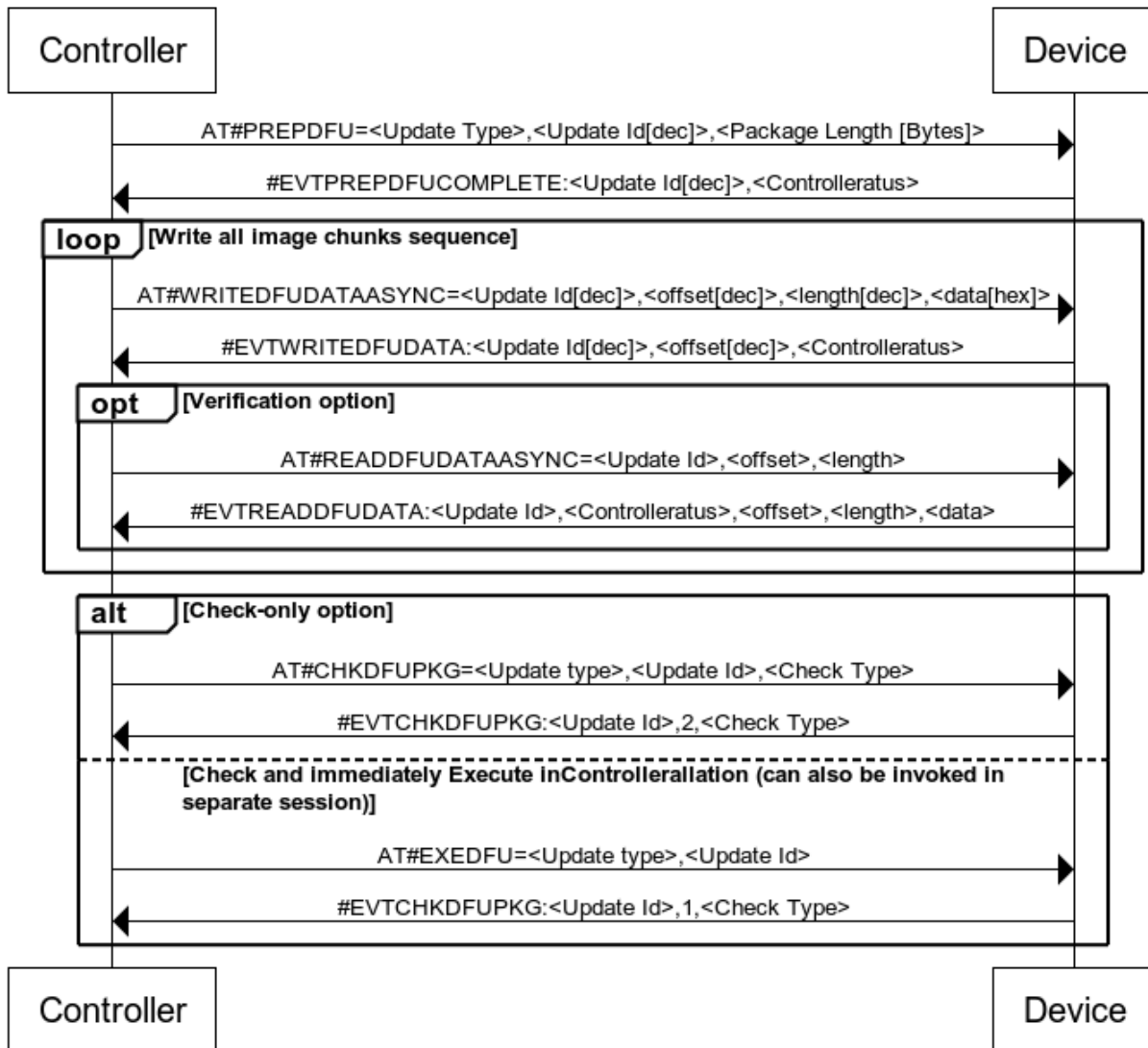
Progress [string] - Human friendly string representing the current progress, which can be progress, error, warning, etc.

During the update there will be a reset, and so during the boot of the new application there will also be regular indication of application start-up.

1.

AT-Commands full update flow sequence diagram

DFU Flow



www.websequencediagrams.com

Sources to the image are stored [here](#).

Debug Commands

Mesh forward message event - #EVTFWDMSG

#EVTFWDMSG: <Source ID[2 bytes]>, <Destination ID[2 bytes]>, <TTL[1 bytes]>,<User Message>

Indication that a message that was not intended for this device id had been received, and now being forwarded.

These messages are usually supposed to be hidden, but can be shown in the case of TBD

The TTL value shown in this message is the value after subtraction. So a message with a value TTL = 0 is only received and not forwarded.

Start simulating unsolicited messages - #STARTSIMRCVMSG

These commands are features to simulate received messages as if they are coming from Mesh side. This can be used for integration and testing with the device communicating via the AT commands interface..

AT#STARTSIMRCVMSG=<Source ID[2 bytes]>,<message length>,<number of messages>,<interval between messages (mSec)>

valid values:

message length: 1-153

number of messages: 0, 1 -255

0 - sending messages nonstop

The value OK is returned in case of valid value or ERROR in case of invalid value.

The device will start sending unsolicited responses #EVTMESHRCV with consecutive numbers 0-99 in the message field, with specified ID address, like it's coming from that device.

Notice that several #STARTSIMRCVMSG commands can coexist and run in parallel. This could be used to simulate multiple connections to several slave devices sending data in parallel.

For example:

sending 3 messages with 6 bytes message len every 0.5 second from a device with ID of 5A22.

```
AT#STARTSIMSLAVMSG=5A22,6,3,500
```

```
OK
```

```
#EVTMESHRCV:5A22,000102030405
```

```
(500 mSec delay)
```

```
#EVTMESHRCV:5A22,060708091011
```

```
(500 mSec delay)
```

#EVTMESHRCV:5A22,121314151617

2. Invalid value (message length = 0) was sent

AT#STARTSIMRCVMSG=5A22,0,3,500,3

ERROR

Stop simulating unsolicited responses - #STOPSIMRCVMSG

AT#STOPSIMRCVMSG=<Source ID[2 bytes]>

The device will stop sending simulated unsolicited messages from specific ID address, and value OK is returned.

Commands for load status over AT-Commands

Get state of main transmitting storage - #HEAPGETSTAT

AT#HEAPGETSTAT=?

#HEAPGETSTAT=<HEAP_ALLOCATED[DEC]>,<HEAP_FREE[DEC]>

OK

<HEAP_ALLOCATED[DEC]>

Currently allocated messages in main transmitting storage(In bytes), print 18 is a value for an empty storage.

<HEAP_FREE[DEC]>

The overall space for messages in main transmitting storage.

All the message inside the system saved at that place in the memory that the packets are waiting to transmit (create or transmit per unit), the command will discover how much storage exists and available refer to every message's income to prevent overload.

Note that the values are not fully similar to the context of message, it will be more than what we send(because of the frame of all message)

Example:

AT#HEAPGETSTAT?

#HEAPGETSTAT=18,15982

OK

Get state of FIFO - #FIFOGETSTAT

```
AT#FIFOGETSTAT=?  
#FIFOGETSTAT=<FULLNESS[0:8][DEC]>  
OK
```

Local temporary storages, printing all the uses FIFO(9 for now), use for internal debug for deep exploration.

```
<com1_fifoRx, com1_fifoTx, uartRx, uartTx, adv, to_receiver_cache, rcv->mesh, atcmd->mesh,  
at_client>
```

```
AT#FIFOGETSTAT?  
#FIFOGETSTAT=8,1214,11,1390,0,0,0,0,0  
OK
```